챕터 3. 도메인을 중심으로 - DDD + 클린 아키텍처

며칠 뒤, 팀장이 오픈이의 자리로 찾아왔습니다.

팀장 : "주문 기능 하나 테스트하려고 하는데, 컨트롤러가 서비스에 직접 의존해서 가짜(Mock) 객체로 테스트할 수가 없어요. 이거 직접 의존하지 않게 구조 정리해 줘요."

오픈이는 곧바로 코드를 열어 봤습니다. 컨트롤러와 서비스가 직접 의존하다 보니, 컨트롤러를 테스트하려면 서비스까지 함께 동작해야 했습니다. 그리고 가짜(Mock) 객체를 넣으려 해도 컨트롤러 코드를 직접 고쳐야 했습니다.

고민에 빠진 오픈이는 선배 자리로 찾아가 코드를 보여 주며 물었습니다.

선배 : "지금 구조는 결합도가 너무 높아서 그래요. 해결하려면 두 가지를 알아야 해요.

첫째는 구현이 아니라 **인터페이스에 의존**하게 만드는 거예요. 서비스가 다른 객체를 직접 호출하지 않고, 둘 사이에 인터페이스를 두는 거죠. 그래야 코드를 고치지 않고도 진짜 객체 대신 가짜 객체로 테스트할 수 있어요. 이게 **클린 아키텍처**의 기본이에요.

둘째는 **도메인 주도 개발(DDD)** 이에요. 핵심 비즈니스 로직을 외부 환경에 두지 말고 도메인 내부에 모아 둬야 해요. 그래야 외부가 어떻게 바뀌든 영향을 받지 않고, 테스트도 도메인만 독립적으로 깔끔하게 끝낼 수 있어요."

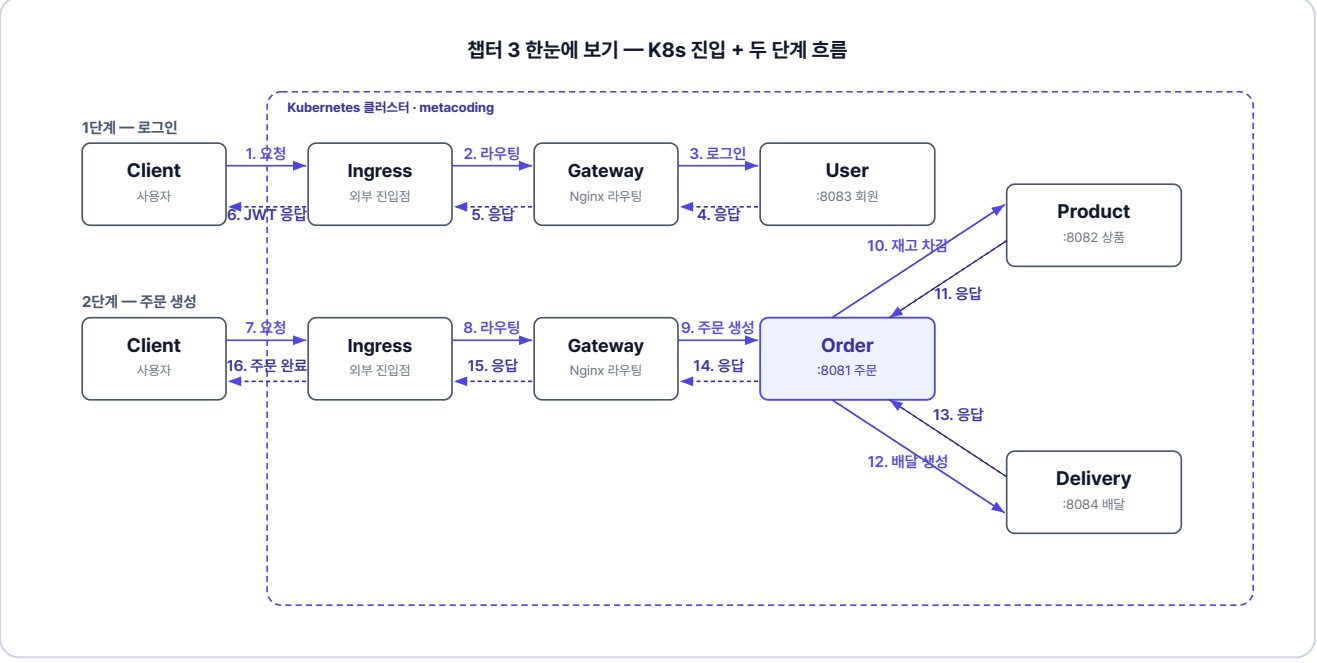


그림 3-1. 챕터 3 한눈에 보기 - K8s 진입과 두 단계 흐름

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex02
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex02` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex02 디렉토리

```
ex02/
├── db/                 # MySQL Dockerfile
├── delivery/           # 포트 8084
├── gateway/            # Nginx API Gateway
├── k8s/                # Kubernetes YAML 파일
├── order/              # 포트 8081
├── product/            # 포트 8082
└── user/               # 포트 8083
```

주문 서비스 패키지 구조 (챕터 3에서 재구성)

```
src/main/java/com/metacoding/order/
├── adapter/        # 외부 서비스 클라이언트 (order 전용)
├── core/           # JWT, 예외처리 (챕터 2와 동일)
├── domain/         # 엔티티 + 비즈니스 규칙
├── repository/     # Spring Data JPA
├── usecase/        # UseCase 인터페이스 + 서비스 코드
└── web/            # 컨트롤러 + DTO
src/main/resources/
└── application.properties        # DB·JWT 설정 (값은 환경변수로 주입)
```

user/product/delivery도 동일한 구조이며, adapter/ 패키지만 order 전용입니다.

3. 실습 환경

도구	용도	비고
Docker Desktop	컨테이너 런타임	챕터 2에서 설치한 그대로. 실행 중이어야 함
Minikube	로컬 Kubernetes 클러스터	https://minikube.sigs.k8s.io/

4. 실습 순서

1. 챕터 2 코드를 DDD + 클린 아키텍처로 재구성하는 과정 살펴보기
2. 주문 서비스에 UseCase 인터페이스 + 도메인 캡슐화 적용
3. Nginx API Gateway와 MySQL 인프라 살펴보기
4. Kubernetes YAML 파일(ConfigMap·Secret·Deployment·Service·Ingress) 5종 살펴보기
5. Minikube에서 빌드·배포·실행

3.1 도메인 주도 개발(Domain-Driven Design) - 비즈니스 로직을 도메인으로

가게 운영 규칙이 **사장 한 명의 머릿속**에만 있다고 해보겠습니다. 환불 가능 시간, 결제 방식, 재고 처리까지 전부 사장이 외우고 있습니다. 그래서 누가 손님을 응대하든 사장이 대답을 해야 합니다.

이 문제는 **가게 운영 매뉴얼**을 만들면 해결됩니다. 규칙은 매뉴얼에 정리해 두고, 사장은 손님 응대 흐름만 진행하면서 매뉴얼에 적힌 대로 따릅니다. **누가 응대하든 매뉴얼만 보면 같은 판단을 할 수 있습니다.** 새 규칙이 들어와도 **매뉴얼 한 곳만 업데이트**하면 끝입니다.

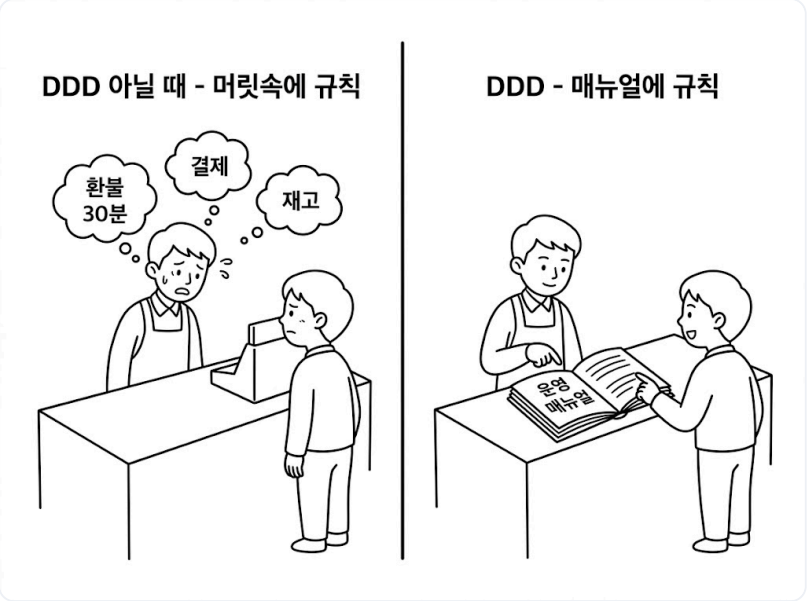


그림 3-2. 머릿속 규칙에서 운영 매뉴얼로

여기서 `OrderService`가 비즈니스를 수행하는 **사장님**이라면, `Order` 도메인 객체는 그 안에 담긴 핵심 규칙인 **운영 매뉴얼**입니다. **비즈니스 로직을 서비스에서 분리해 도메인에 모아 두면** 기술적 환경이 변해도 비즈니스의 본질은 영향을 받지 않으며, 복잡한 요구사항 속에서도 코드의 가독성과 유지보수성을 지킬 수 있습니다.

3.2 UseCase 인터페이스(Use Case Interface) - USB 허브

비즈니스를 수행하는 서비스가 컨트롤러에 직접 묶여 있으면 두 코드가 **강하게 결합됩니다.** 그래서 서비스를 고칠 때마다 컨트롤러도 함께 고쳐야 하고, 테스트할 때 진짜 서비스 대신 **가짜(Mock) 객체를 끼워 넣기도 불가능**해집니다.

이 문제를 해결하려면 컨트롤러와 서비스 사이에 **느슨한 연결 고리**가 필요합니다. 컴퓨터와 USB 허브를 떠올려 보세요. 여러 장치를 컴퓨터에 직접 연결하지 않고 USB 허브를 거쳐 연결하면, 뒤에서 장치를 아무리 바꾸더라도 **컴퓨터와 허브 사이의 연결에는 아무런 변화가 없습니다.**

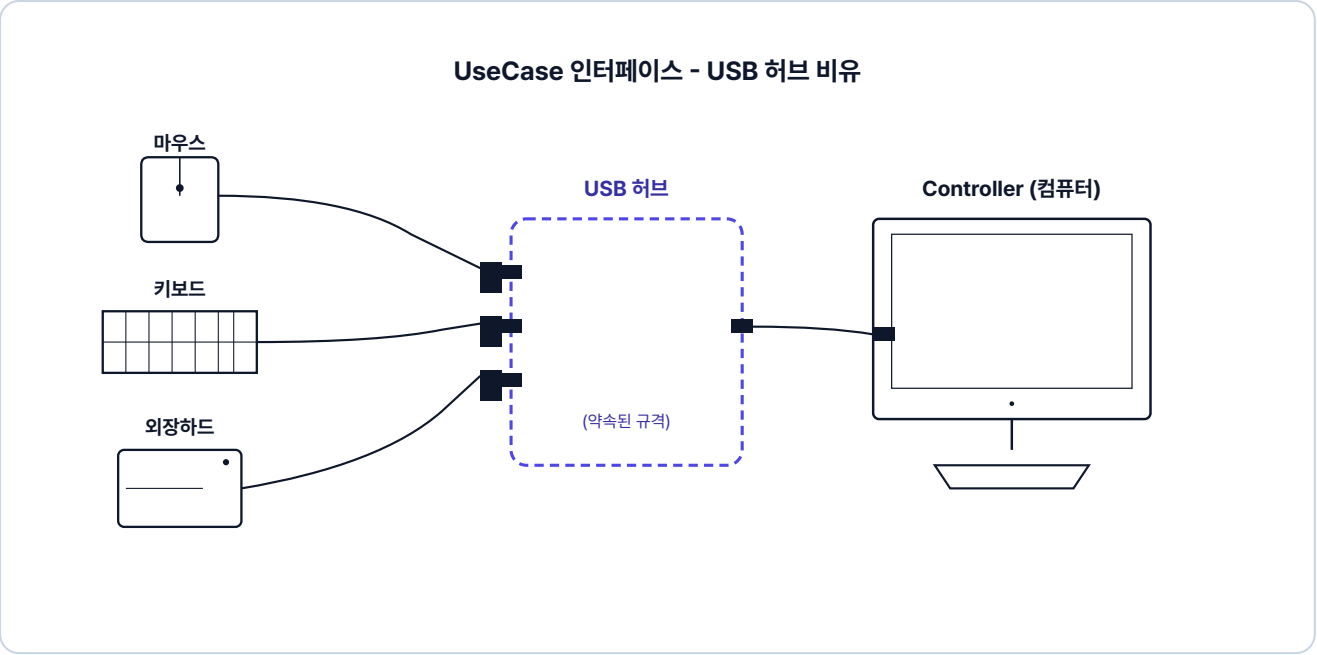


그림 3-3. UseCase 인터페이스 - USB 허브 비유

여기서 USB 허브의 역할을 하는 것이 **UseCase 인터페이스**입니다.

> **UseCase 인터페이스란?** 시스템이 수행할 비즈니스 **행위(주문 생성·조회·취소 같은)** 를 메서드로 약속한 인터페이스입니다.

컨트롤러가 구현체 대신 UseCase 인터페이스를 참조하고, 서비스가 이 인터페이스를 구현하면 둘은 독립적으로 동작합니다. 이렇게 의존 관계를 느슨하게 만들어 내부 로직을 보호하는 것이 **클린 아키텍처**의 원칙입니다.

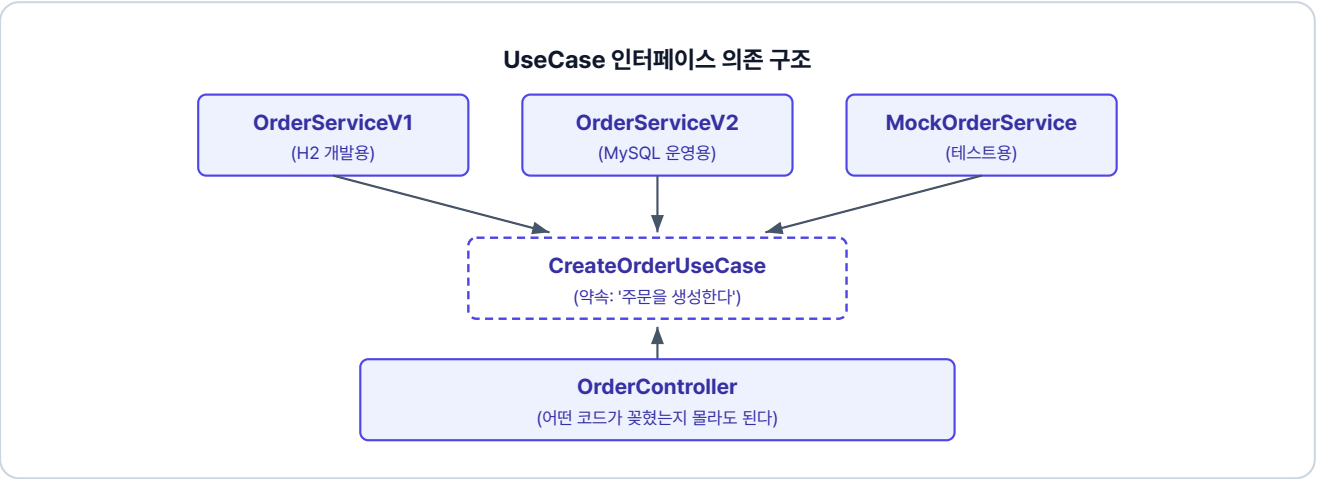


그림 3-4. UseCase 인터페이스 의존 구조

"무엇을 할 것인가"(UseCase 인터페이스) 와 **"어떻게 할 것인가"(Service 코드)** 를 분리하는 것이 핵심입니다.

3.3 패키지 구조 비교 - 단일 패키지에서 책임별 패키지로

챕터 2는 **단일 패키지 구조**입니다. 주문과 관련된 모든 클래스가 `orders/` 한 폴더에 모여 있습니다.

반면 챕터 3은 **책임별 패키지 구조**입니다. 같은 주문 코드가 역할에 따라 네 폴더로 나뉩니다. 앞에서 다룬 도메인 응집과 인터페이스 분리가 폴더 구조에 그대로 반영됩니다.

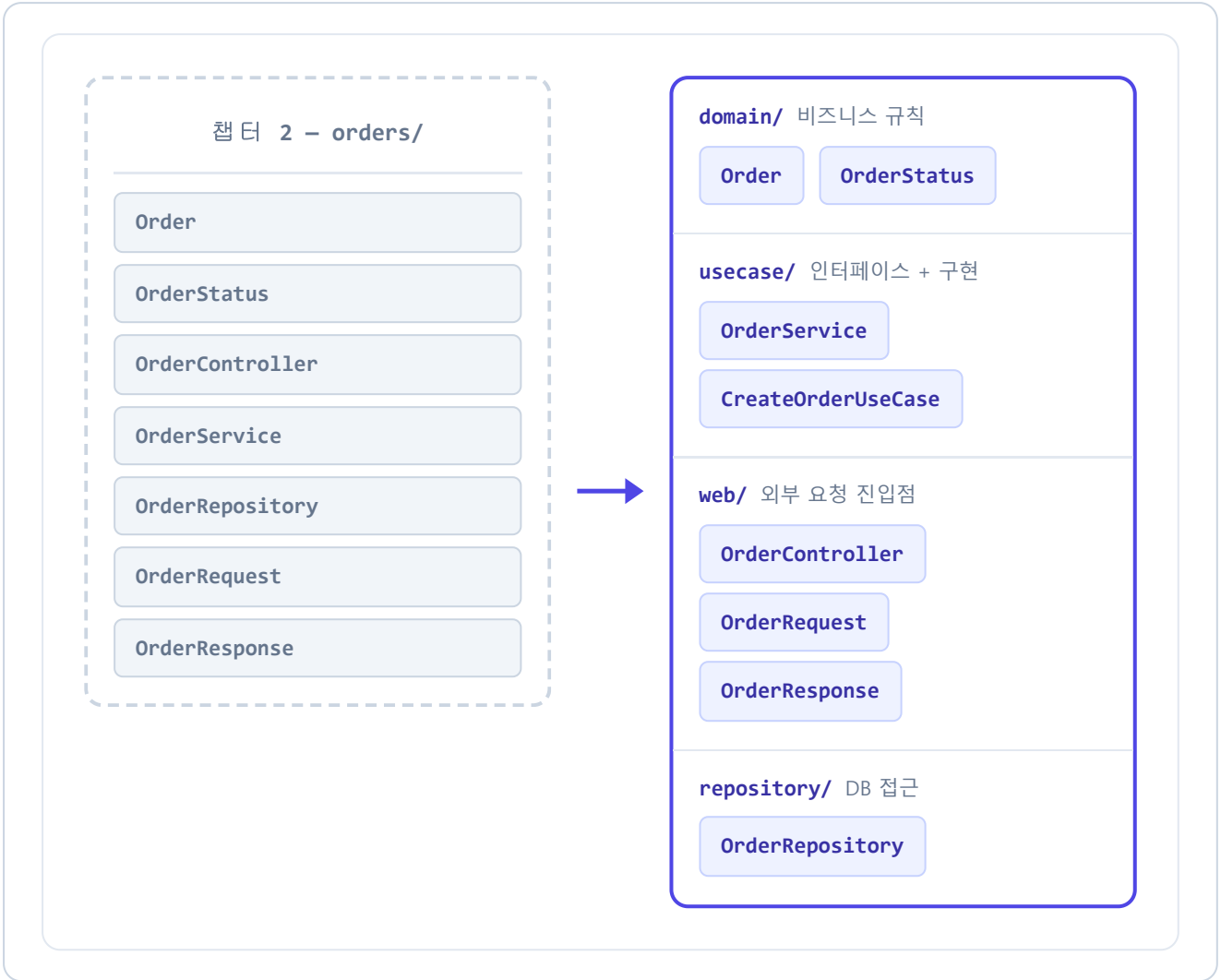


그림 3-5. 패키지 구조 비교 - 단일 패키지에서 책임별 패키지로

이렇게 도메인을 중심에 두고 외부 의존을 인터페이스로 분리하는 패키지 구조를 **헥사고날 패턴 (Hexagonal Architecture)** 이라고 합니다. 이 책에서는 완전한 아키텍처보다는 실습에 필요한 개념만 적용합니다.

3.4 UseCase 인터페이스 + 도메인 캡슐화 도입

이제 실제 코드에 적용해보겠습니다.

3.4.1 UseCase 인터페이스 정의

주문·상품·회원·배달 서비스의 각 기능을 인터페이스 형태로 UseCase로 정의합니다. **인터페이스 하나가 하나의 행위(Use Case)를 의미합니다.**

주문 서비스의 `usecase/CreateOrderUseCase.java`를 열고 주문 생성 인터페이스를 작성합니다.

실습 1 **주문 서비스 - usecase/CreateOrderUseCase.java. 주문 생성 인터페이스**

```
// 주문을 생성한다 - 행위 하나를 인터페이스로 약속
public interface CreateOrderUseCase {
    OrderResponse createOrder(int userId, int productId,
            int quantity, Long price, String address);
}
```

조회는 `GetOrderUseCase`, 취소는 `CancelOrderUseCase`로 같은 방식으로 정의합니다.

3.4.2 엔티티의 비즈니스 로직 — DDD의 핵심

"주문 금액이 최소 기준을 넘는가?" 같은 비즈니스 규칙은 서비스가 아닌 엔티티에 둡니다. 엔티티 메서드로 캡슐화하면 어디서 호출하든 동일한 규칙이 적용되고, 새 규칙이 들어와도 도메인 메서드만 추가하면 됩니다.

주문 서비스의 `domain/Order.java`를 열고 최소 주문 금액 검증 메서드를 작성합니다.

실습 2 **주문 서비스 - domain/Order.java. 비즈니스 규칙을 도메인에 캡슐화**

```
public class Order {
    // 챕터 2 Order.java 참조 — 필드·create()·complete() 동일

    // 최소 주문 금액 검증 (챕터 2에서 서비스에 있던 검증을 도메인으로 옮김)
    public void validateMinAmount() {
        if (this.quantity * this.price < 1000) {
            throw new Exception400("최소 주문 금액은 1,000원입니다.");
        }
    }
```

```
}
```

3.4.3 OrderService - 인터페이스 구현

OrderService는 주문 생성, 주문 조회, 주문 취소 인터페이스를 구현하고, 도메인 객체의 비즈니스 메서드를 호출합니다.

주문 서비스의 `usecase/OrderService.java`를 열고 UseCase 인터페이스 구현을 작성합니다.

실습 3 주문 서비스 - usecase/OrderService.java. UseCase 인터페이스 구현

```
@RequiredArgsConstructor
@Service
@Transactional(readOnly = true)
public class OrderService
        implements CreateOrderUseCase, GetOrderUseCase, CancelOrderUseCase {
    // 1. UseCase 인터페이스를 구현

    @Override
    @Transactional
    public OrderResponse createOrder(int userId, int productId,
            int quantity, Long price, String address) {
        Order createdOrder = orderRepository.save(
                Order.create(userId, productId, quantity, price));
        // 2. 검증 메서드 호출
        createdOrder.validateMinAmount();
        // ... 재고 차감 → 배달 생성 → 완료 (보상 트랜잭션)
    }
}
```

3.4.4 OrderController 수정

컨트롤러는 서비스가 아닌 인터페이스에 의존합니다. `OrderService`를 다른 코드로 바꿔도 이 컨트롤러는 전혀 수정할 필요가 없습니다.

주문 서비스의 `web/OrderController.java`는 아래처럼 인터페이스에 의존하도록 작성되어 있습니다.

참고 주문 서비스 - web/OrderController.java. UseCase 인터페이스 주입

```
@RestController
@RequestMapping("/api/orders")
```

```java
@RequiredArgsConstructor
public class OrderController {
    private final CreateOrderUseCase createOrderUseCase;   // 인터페이스 주입
    private final GetOrderUseCase getOrderUseCase;
    private final CancelOrderUseCase cancelOrderUseCase;

    @PostMapping
    public ResponseEntity<?> createOrder(...) {
        return Resp.ok(createOrderUseCase.createOrder(...));  // 인터페이스 메서드 호출
    }

    // GET /{orderId} — 주문 조회
    // PUT /{orderId} — 주문 취소
}
```

주문 서비스와 동일한 패턴으로 상품, 배달, 회원 서비스도 구성되어 있습니다.

내부 구조를 정리했으니, 이제 네 서비스를 운영 환경에 올릴 차례입니다.

오픈이 : "그런데 지금까지는 요청할 때마다 서비스 포트를 바꿔야 했어요. 이건 어떻게 해결하나요?"

선배 : "내부를 수정했으니, 외부에서 들어오는 단일 진입점도 만들면 좋겠네요."

3.5 Gateway와 MySQL 인프라

3.5.1 Nginx - API Gateway 라우팅

서비스가 늘어날수록 클라이언트는 모든 포트를 알아야 하고, 서비스 주소가 바뀌면 그때마다 코드를 고쳐야 합니다. 이때 **API Gateway**를 앞에 두면, 클라이언트는 하나의 진입점으로만 요청하고 게이트웨이가 URL 경로에 따라 알맞은 서비스로 전달합니다.

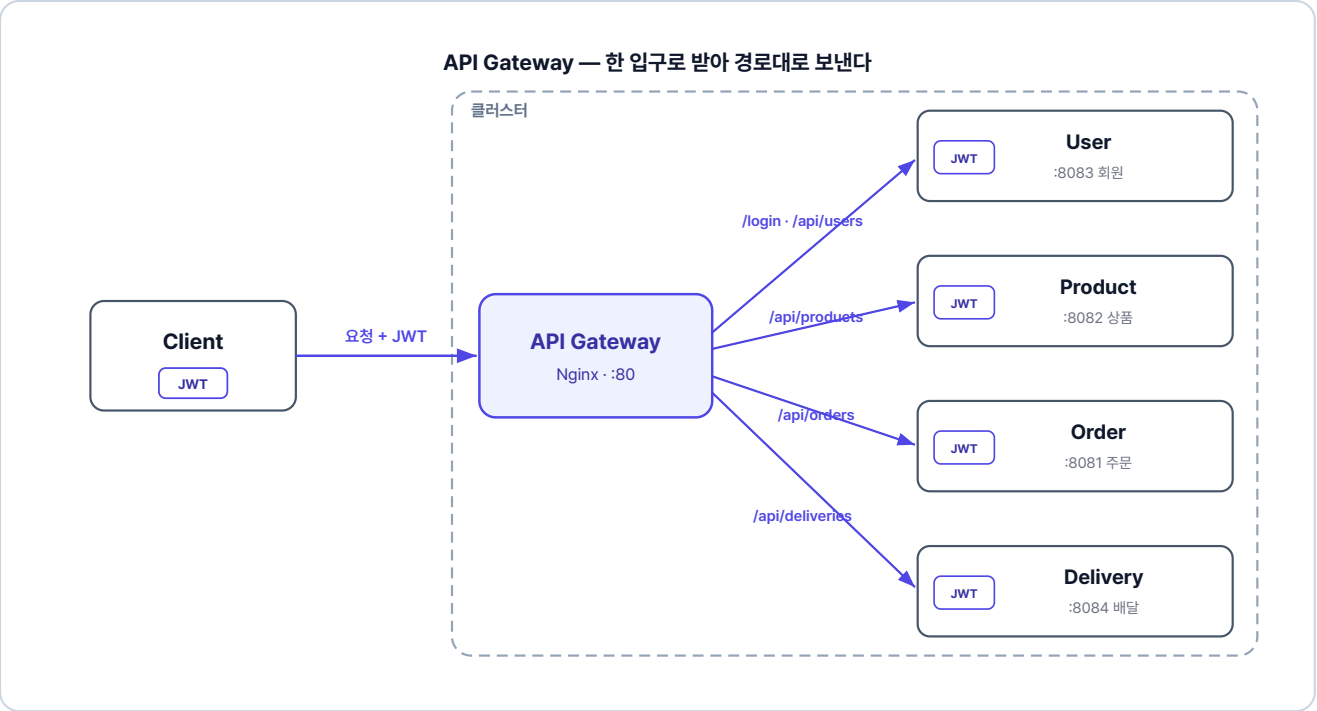


그림 3-6. 한 입구로 받아 경로대로 전달하고, 토큰 검증은 각 서비스가 합니다

`gateway/` 디렉토리에는 두 파일이 있습니다. 전체 설정은 GitHub을 참고하세요.

파일	역할
Dockerfile	Nginx 베이스 이미지에 설정 파일을 넣어 게이트웨이 컨테이너를 만듭니다.
nginx.conf	URL 경로별로 어느 서비스에 요청을 보낼지 정의합니다.

게이트웨이에서 토큰을 검증하는 방식

지금 이 책에서는 게이트웨이(Nginx)가 요청을 경로대로 넘기기만 하고, JWT는 각 서비스가 직접 검증합니다. 다른 방법으로, 게이트웨이가 입구에서 토큰을 먼저 검증하고 통과한 요청만 서비스로 보내는 구조도 널리 쓰입니다. 이때 게이트웨이는 토큰에서 꺼낸 사용자 정보를 헤더에 실어 전달하고, 내부 서비스는 게이트웨이를 지나온 요청을 이미 인증된 것으로 신뢰합니다.

게이트웨이에서 검증하면 인증이 한 곳에 모여 각 서비스는 비즈니스 로직에만 집중할 수 있습니다. 다만 Nginx만으로는 토큰 검증이 어려워 Spring Cloud Gateway 같은 도구가 필요합니다. 이 책은 단순함을 위해 서비스별 검증을 택했습니다.

3.5.2 MySQL - 데이터베이스 인프라

모든 서비스가 동일한 MySQL 인스턴스를 공유합니다. 서비스별로 테이블이 분리되어 있으나, 물리적으로는 단일 DB 인스턴스입니다.

> **이 책에서는 학습 편의를 위해 하나의 DB를 공유합니다.** 실제 MSA에서는 서비스마다 독립된 DB를 두지만, 학습 흐름을 익히는 데는 차이가 없으니 DB 구성보다 흐름에 집중해 주세요.

DB 컨테이너는 `db/` 디렉토리의 두 파일로 구성됩니다.

파일	역할
Dockerfile	MySQL 공식 이미지를 베이스로 초기화 SQL을 컨테이너에 넣어 띄웁니다.
init.sql	네 서비스에 필요한 테이블을 만들고 더미 데이터를 채웁니다.

3.6 Kubernetes - YAML로 선언하는 배포

오픈이 : "준비가 끝났으니, 이제 배포를 시작할까요?"

선배 : "그 전에 실무에 올리려면 한 가지 더 생각해야 해요. 지금은 Docker Compose로 컨테이너를 직접 띄우고 있잖아요? 만약 서버가 멈추거나 컨테이너가 갑자기 내려가면 어떻게 될까요?"

오픈이 : "음... 개발자가 서버에 접속해서 다시 띄워야 하지 않을까요?"

선배 : "사람이 24시간 내내 서버만 지켜볼 수는 없죠. 쿠버네티스는 우리가 원하는 상태를 정해 두면, 컨테이너가 죽더라도 알아서 다시 살려내서 그 상태를 유지해 줘요."

3.6.1 리소스 구조 설계

Kubernetes는 YAML 파일로 원하는 상태를 선언합니다. **"이 서비스는 이렇게 실행되어야 한다"** 고 파일에 적어두면, Kubernetes가 그 상태를 유지합니다.

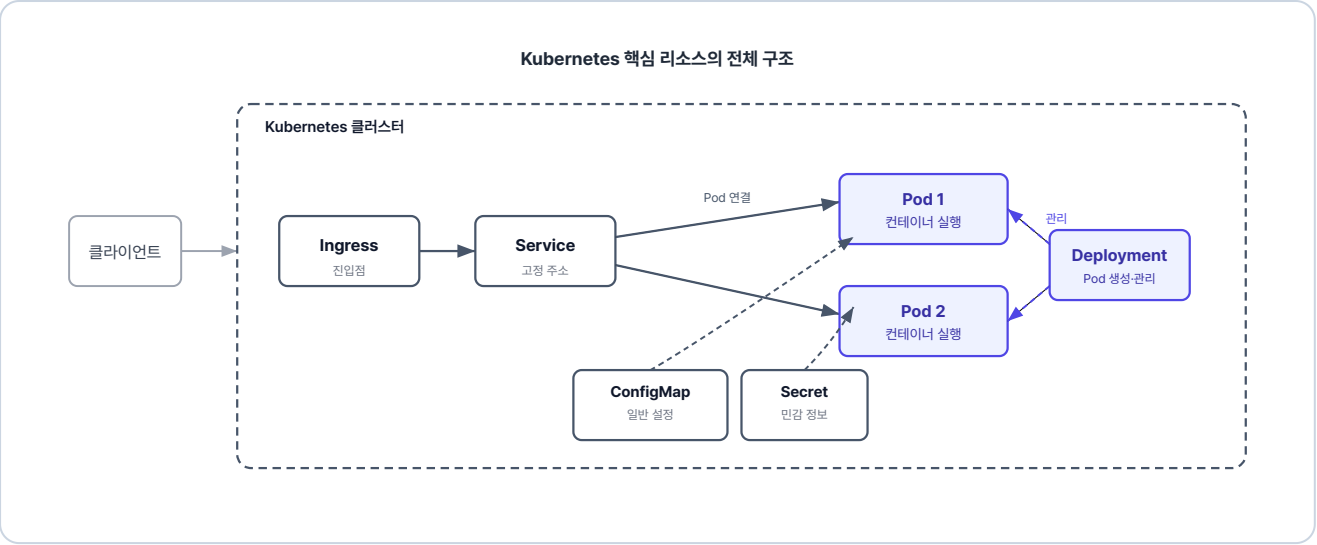


그림 3-7. Kubernetes 리소스 관계

각 Kubernetes 리소스의 역할을 정리하면 다음과 같습니다.

리소스	역할
ConfigMap	일반 환경변수(DB 주소 등)를 외부에서 주입합니다.
Secret	DB 계정·비밀번호 같은 민감 정보를 분리해 관리합니다.
Deployment	Pod를 어떻게 실행할지 정의하고, ConfigMap과 Secret을 한꺼번에 주입합니다.
Service	Pod에 고정 주소를 부여해 클러스터 안에서 안정적으로 통신할 수 있게 합니다.
Ingress	클러스터 외부 요청을 클러스터 안으로 들여보냅니다.

나머지 서비스(product, user, delivery)도 동일한 패턴입니다. 전체 YAML 파일은 레포의 `k8s/` 디렉토리를 참고하세요.

Gateway API로 두면 로컬 설정을 클라우드로 그대로 옮기기 쉽다

이 책은 외부 진입을 Ingress로 구성하지만, 후속 표준인 **Gateway API(우리가 만든 Gateway와 다름)** 를 쓰는 방법도 있습니다. Gateway API는 외부 진입점과 경로 규칙을 따로 정의합니다.

이렇게 나뉘어 있으면 로컬에서 클라우드로 옮길 때 편합니다. 경로 규칙은 그대로 두고, 진입점만 환경에 맞게 바꾸면 됩니다. 게다가 AWS EKS 같은 클라우드에서는 진입점조차 직접 만들 필요가 없습니다. 클라우드가 Gateway API 설정을 읽어 로드 밸런서를 자동으로 붙여 주기 때문입니다.

3.7 Minikube - 실행 및 결과 확인

3.7.1 Minikube 시작

Minikube는 로컬 PC에 가벼운 Kubernetes 클러스터를 만들어주는 도구입니다. 설치되어 있지 않다면 OS에 맞게 먼저 설치합니다.

터미널 **Minikube 설치**

```
# macOS
brew install minikube

# Windows
winget install Kubernetes.minikube
```

설치한 뒤 새 터미널을 열고, Docker Desktop이 실행 중인 상태에서 아래 명령을 입력하면 클러스터가 생성됩니다.

터미널 **Minikube 시작**

```
minikube start
```

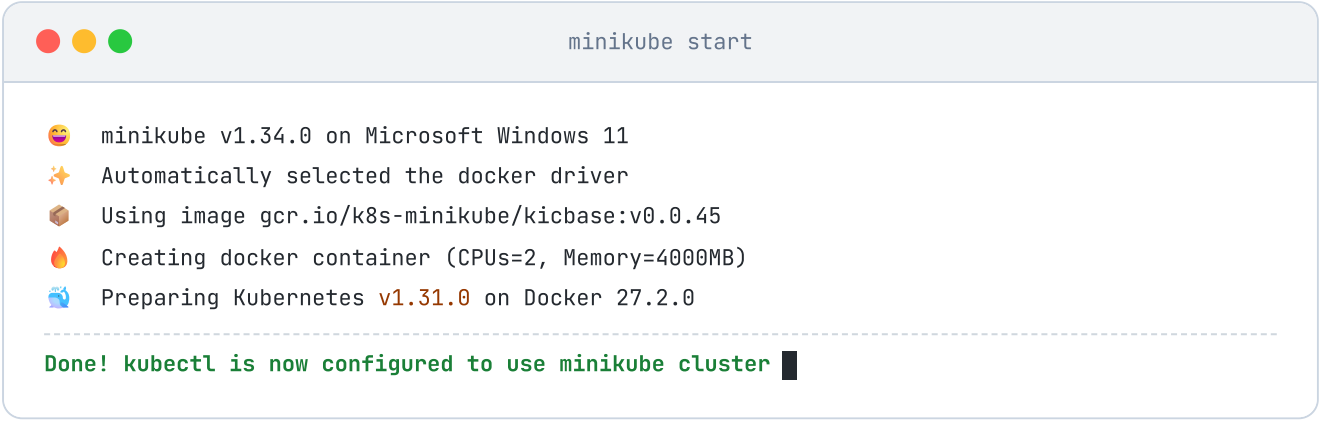


그림 3-8. Minikube 시작

3.7.2 이미지 빌드

`minikube image build`는 Minikube 내부에 직접 이미지를 빌드합니다.

터미널 **이미지 빌드**

```
minikube image build -t metacoding/db:1 ./db
minikube image build -t metacoding/order:1 ./order
minikube image build -t metacoding/product:1 ./product
minikube image build -t metacoding/user:1 ./user
minikube image build -t metacoding/delivery:1 ./delivery
minikube image build -t metacoding/gateway:1 ./gateway
```

minikube image build · 6개 서비스

```
→ Successfully tagged metacoding/db:1
→ Successfully tagged metacoding/order:1
→ Successfully tagged metacoding/product:1
→ Successfully tagged metacoding/user:1
→ Successfully tagged metacoding/delivery:1
→ Successfully tagged metacoding/gateway:1

6개 이미지 빌드 완료
```

그림 3-9. 이미지 빌드 결과

3.7.3 배포 순서

네임스페이스를 먼저 생성하고, DB가 준비된 뒤에 나머지 서비스를 배포합니다.

터미널 **배포 순서**

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. DB 관련 리소스 먼저 배포
kubectl apply -f k8s/db

# 3. 각 서비스 배포
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway

# 4. Ingress 활성화 (Minikube에서는 애드온 활성화 필요)
minikube addons enable ingress
```

kubectl apply · 네임스페이스 + 6개 서비스

```
namespace/metacoding created
secret/db-secret created
deployment.apps/db-deploy created
service/db-service created
configmap/order-configmap created
secret/order-secret created
deployment.apps/order-deploy created
service/order-service created
  … product · user · delivery · gateway 동일 패턴 …
ingress.networking.k8s.io/gateway-ingress created

전체 리소스 배포 완료
```

그림 3-10. 네임스페이스 생성 및 배포

3.7.4 배포 상태 확인

배포가 끝나면 모든 Pod가 제대로 실행되고 있는지 확인합니다.

터미널 **Pod 상태 확인**

```
kubectl get pods -n metacoding
```

kubectl get pods -n metacoding

```
NAME                                READY   STATUS    AGE
db-deploy-6f9b7c4d8-m4t2q           1/1     Running   42s
gateway-deploy-5c8d6f7b9-h7w3r      1/1     Running   38s
order-deploy-8b7f6c9d4-q2k8m        1/1     Running   36s
product-deploy-7c9d8b6f5-x4r2t      1/1     Running   35s
user-deploy-6d8c7b9f4-p3m9k         1/1     Running   33s
delivery-deploy-9f7c8b6d5-t6w2x     1/1     Running   30s

모든 Pod Running
```

그림 3-11. Pod 상태 확인

모든 Pod가 `Running` 상태가 되면 배포 완료입니다.

3.7.5 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel`을 실행합니다.

터미널 **외부 접근 터널**

```
minikube tunnel
```

`minikube tunnel`은 터미널을 점유합니다.

터널이 실행되면 `http://127.0.0.1:80`로 gateway-service에 접속할 수 있습니다. 회원 서비스가 발급한 토큰은 하루 동안 유효하므로, 챕터 2에서 받은 토큰을 그대로 써서 아래 주문을 생성합니다. 만료됐다면 같은 방법으로 다시 발급받습니다.

MacBook Pro(상품 ID 1) 1개를 배달 주소와 함께 주문합니다.

Hoppscotch **주문 생성**

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

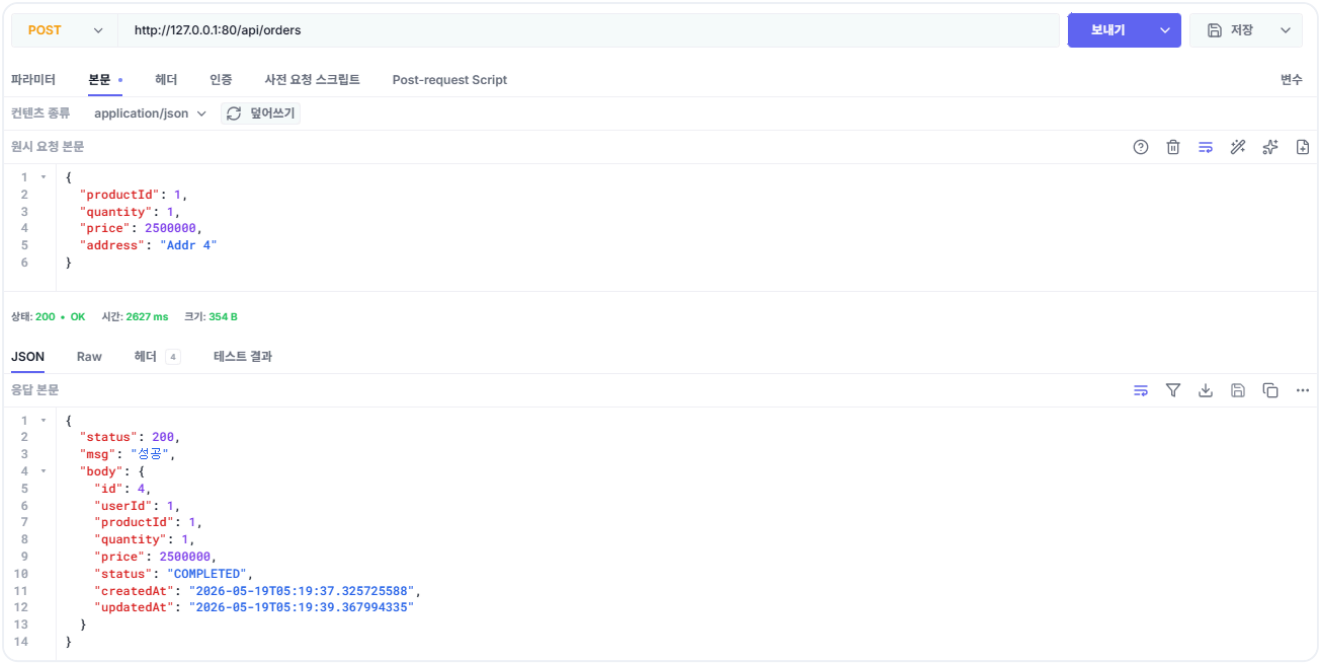


그림 3-12. 주문 결과 확인

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 **리소스 정리**

```
kubectl delete namespace metacoding
```

오픈이 : "쿠버네티스 위에서도 주문이 문제없이 만들어졌어요. 구조도 한결 깔끔해졌어요."

선배 : "맞아요. 이제 결합도가 낮아졌으니, 서비스를 수정해도 컨트롤러는 영향을 받지 않아요."

다음 챕터에서는 동기 호출 방식에서 메시지를 통한 비동기 호출 방식으로 전환합니다.

이것만은 기억하자

- → **DDD**로 비즈니스 규칙을 서비스가 아니라 도메인 객체에 둡니다.
- → **클린 아키텍처**(UseCase 인터페이스)로 컨트롤러가 구현이 아닌 인터페이스에 의존합니다. 덕분에 환경이나 테스트에 따라 구현을 바꿀 수 있습니다.
- → **Nginx API Gateway**로 여러 서비스의 진입점을 하나로 모으고, URL 경로에 따라 알맞은 서비스로 전달합니다.
- → **Kubernetes**로 원하는 상태를 선언하면, 컨테이너가 내려가도 그 상태를 유지합니다.